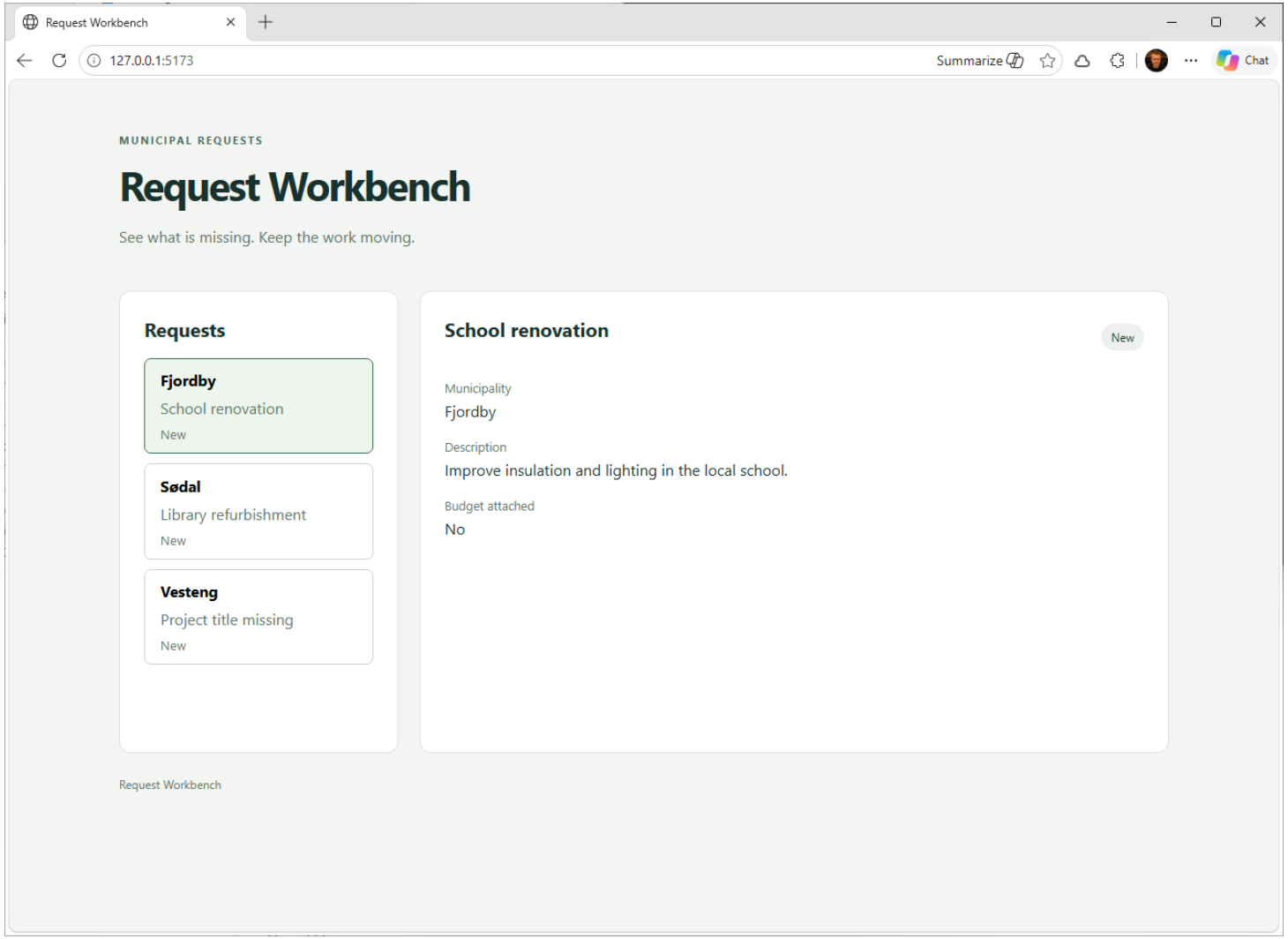
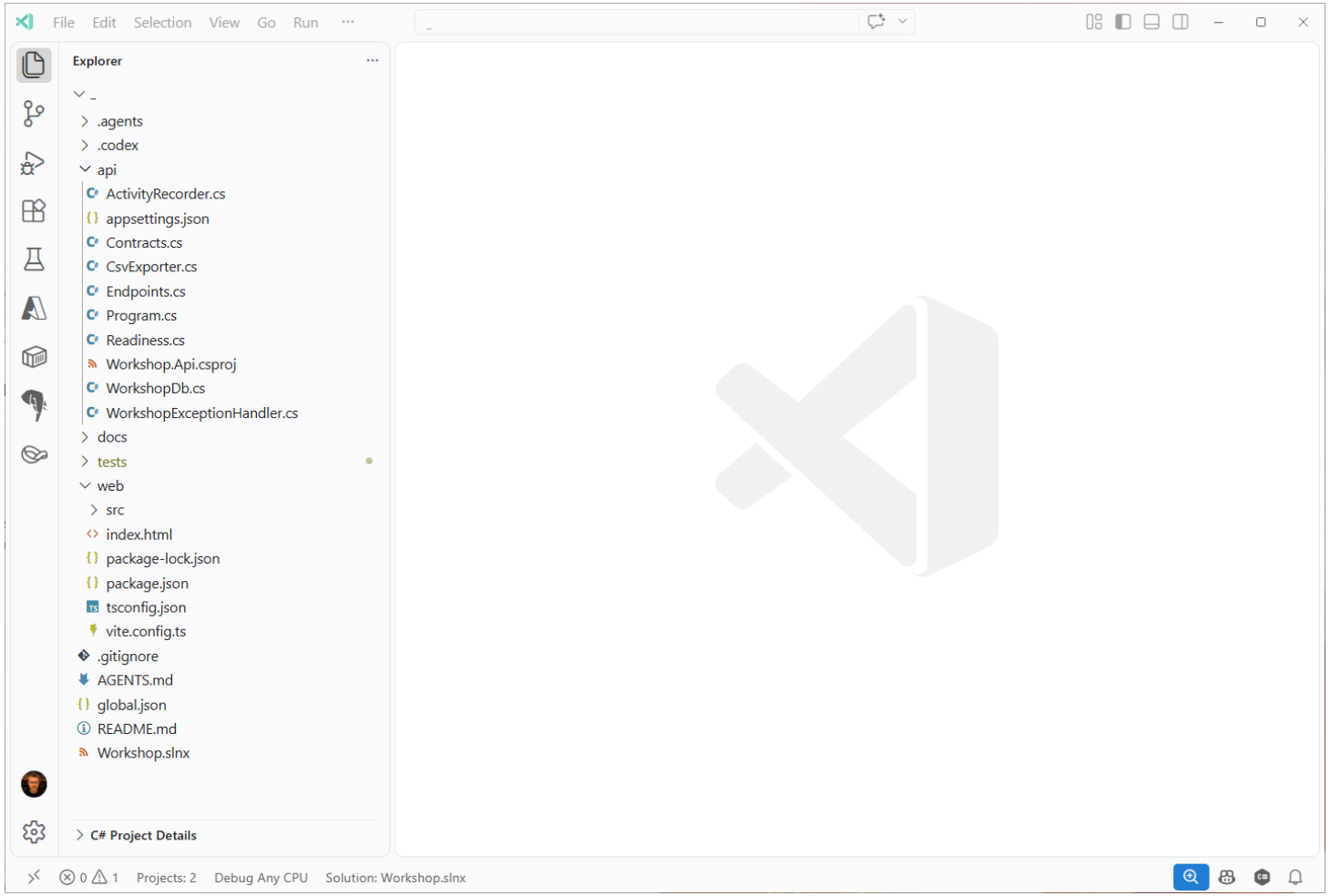
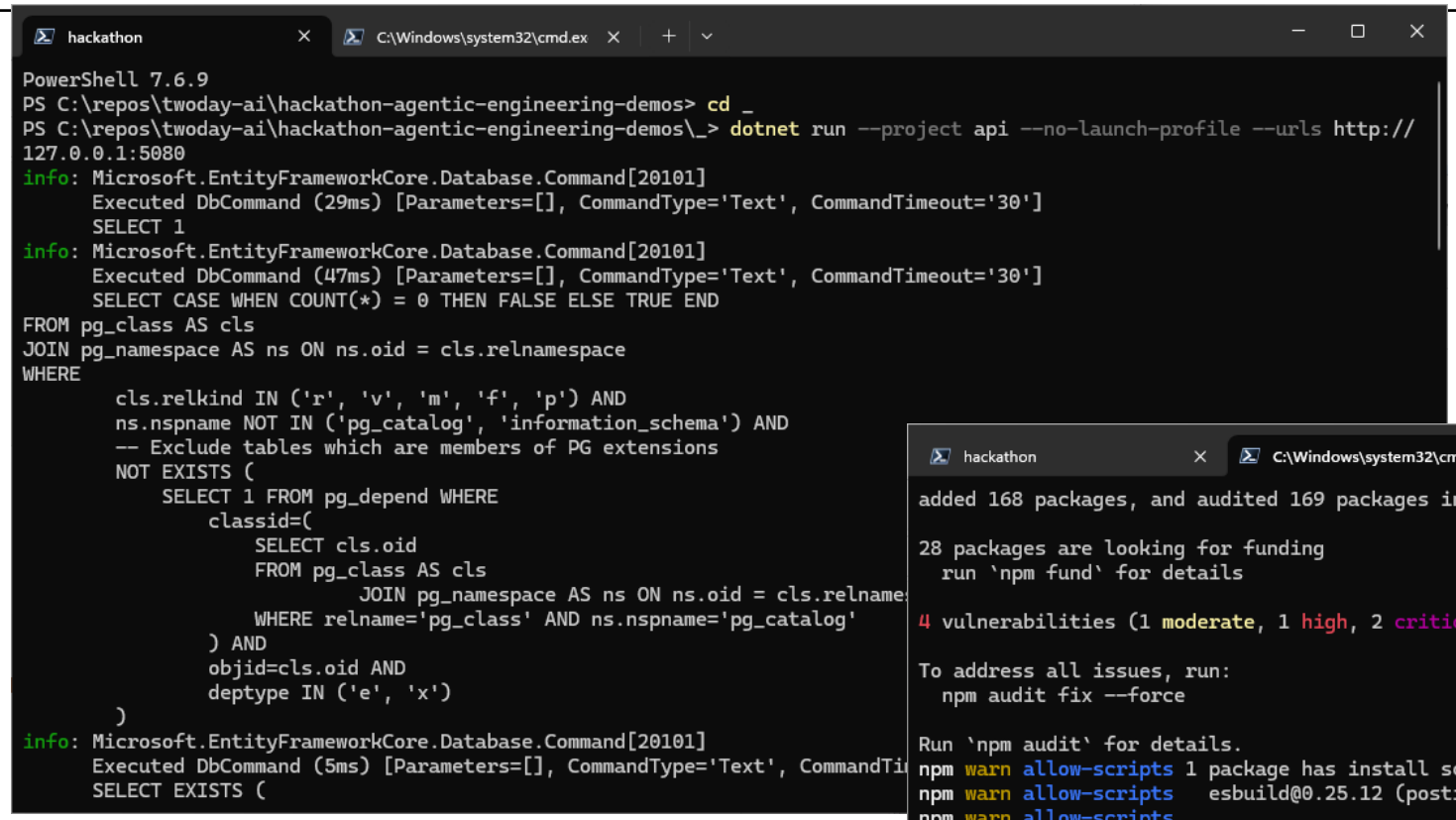


Introduction to Request Workbench

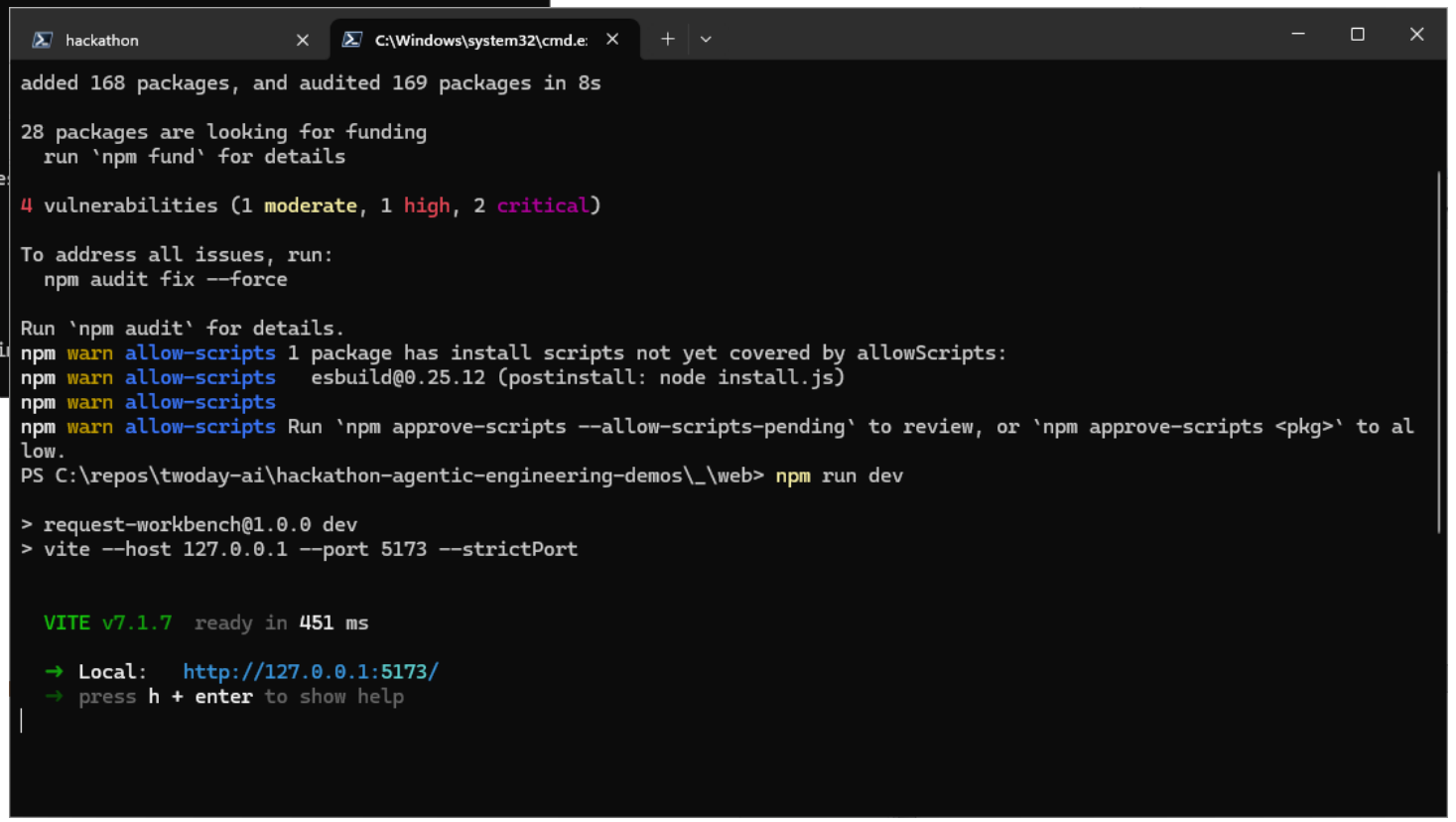

Request Workbench
127.0.0.1:5173
Summarize
Chat
MUNICIPAL REQUESTS
Request Workbench
See what is missing. Keep the work moving.
Requests
Fjordby
School renovation
New
Sødal
Library refurbishment
New
Vesteng
Project title missing
New
School renovation
New
Municipality
Fjordby
Description
Improve insulation and lighting in the local school.
Budget attached
No
Request Workbench


File Edit Selection View Go Run
Explorer
_
.agents
.codex
api
ActivityRecorder.cs
appsettings.json
Contracts.cs
CsvExporter.cs
Endpoints.cs
Program.cs
Readiness.cs
Workshop.Api.csproj
WorkshopDb.cs
WorkshopExceptionHandler.cs
docs
tests
web
src
index.html
package-lock.json
package.json
tsconfig.json
vite.config.ts
.gitignore
AGENTS.md
global.json
README.md
Workshop.slnx
C# Project Details
0 1
Projects: 2
Debug Any CPU
Solution: Workshop.slnx


hackathon
C:\Windows\system32\cmd.ex
PowerShell 7.6.9
PS C:\repos\twoday-ai\hackathon-agentic-engineering-demos> cd _
PS C:\repos\twoday-ai\hackathon-agentic-engineering-demos_> dotnet run --project api --no-launch-profile --urls http://
127.0.0.1:5080
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (29ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (47ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT CASE WHEN COUNT(*) = 0 THEN FALSE ELSE TRUE END
FROM pg_class AS cls
JOIN pg_namespace AS ns ON ns.oid = cls.relnamespace
WHERE
cls.relkind IN ('r', 'v', 'm', 'f', 'p') AND
ns.nspname NOT IN ('pg_catalog', 'information_schema') AND
-- Exclude tables which are members of PG extensions
NOT EXISTS (
SELECT 1 FROM pg_depend WHERE
classid=(
SELECT cls.oid
FROM pg_class AS cls
JOIN pg_namespace AS ns ON ns.oid = cls.relname
WHERE relname='pg_class' AND ns.nspname='pg_catalog'
) AND
objid=cls.oid AND
deptype IN ('e', 'x')
)
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
Executed DbCommand (5ms) [Parameters=[], CommandType='Text', CommandTi
SELECT EXISTS (

API

Web


hackathon
C:\Windows\system32\cmd.e:
added 168 packages, and audited 169 packages in 8s
28 packages are looking for funding
run `npm fund` for details
4 vulnerabilities (1 moderate, 1 high, 2 critical)
To address all issues, run:
npm audit fix --force
Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts esbuild@0.25.12 (postinstall: node install.js)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to al
low.
PS C:\repos\twoday-ai\hackathon-agentic-engineering-demos_\web> npm run dev
> request-workbench@1.0.0 dev
> vite --host 127.0.0.1 --port 5173 --strictPort
VITE v7.1.7 ready in 451 ms
Local: http://127.0.0.1:5173/
press h + enter to show help

/case/readme.md

Request Workbench — Product owner handover

From Product Owner, Municipal Requests
To Project Manager and the developer joining the Request Workbench team

Hi both,

Welcome to the team. Request Workbench is where our caseworkers pick up project requests that municipalities have already sent us. The caseworkers open a request, check that the information we need is there, correct the record as details come in, and mark it **Reviewed** when the intake check is complete. The operations lead uses that status to see which requests are still waiting for attention. A review is a check that the request is complete for intake. The funding decision happens elsewhere.

You're joining with a working request list and a read-only detail view. Use the current `case/start-here` checkout and its sample requests. The README in that folder explains how to run it. We want to build on what's there. The list is how caseworkers find their work today, and they'll still need to move between requests and see which are **New** and which are **Reviewed**.

/case/case-getting-ready.md

Workshop getting started

Set up Request Workbench before the first build session. Both the step-by-step and open-ended tracks use the same starter project, sample data, and checks. The steps below assume Windows and PowerShell.

1. Get the tools and the starter

Tool	What to have ready
Git	Access to the workshop repository if you're cloning it
.NET SDK	.NET 10 SDK, version 10.0.100 or a later .NET 10 feature band
Node.js and npm	Node.js 22.12 or newer, with npm
PostgreSQL	A local Windows installation with the server running and the PostgreSQL command-line tools available
VS Code or VS Code Insiders	C# tooling, a PostgreSQL extension if you prefer to run SQL in the editor, and Codex signed in

From a new PowerShell terminal, check that the tools are available.

```
git --version
dotnet --list-sdks
node --version
npm --version
psql --version
```

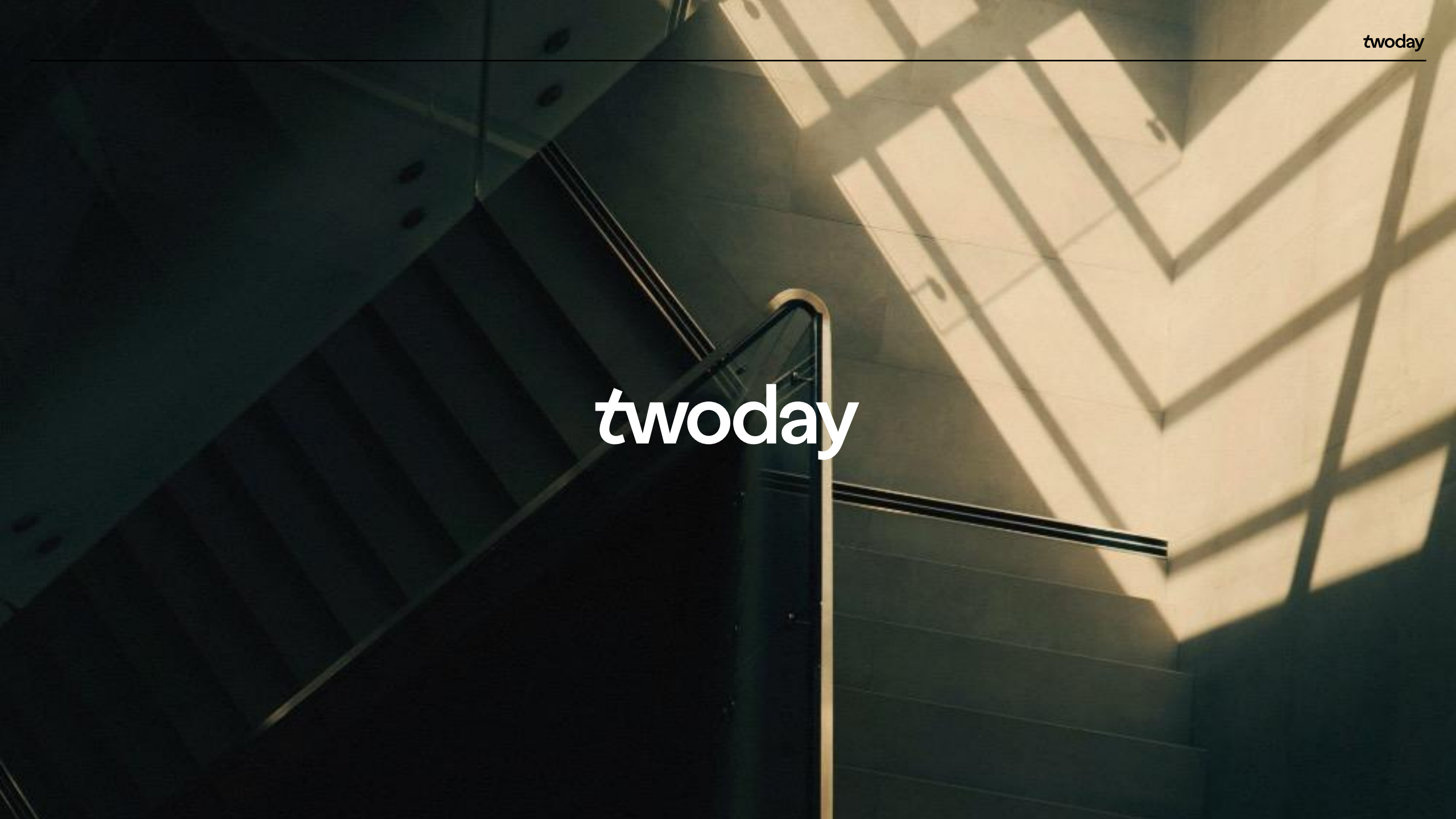
Get familiar with the existing app

```
Explain this unfamiliar application to a new engineer.

Identify users implied by the UI, main capabilities, entry points, important business processes, and external
dependencies evidenced in the repository.

Trace one list → details → edit → review journey where supported. Cite files/symbols. Mark unknowns.

Do not edit and do not create or add fictional rules.
```


twoday